arXiv:2506.04962v4 [cs.CR] 28 Jun 2026

PoCGen: Generating Proof-of-Concept Exploits for Vulnerabilities in Npm Packages

DENIZ SIMSEK, University of Stuttgart, Germany
ARYAZ EGHBALI, CISPA Helmholtz Center for Information Security, Germany
MICHAEL PRADEL, CISPA Helmholtz Center for Information Security, Germany

Security vulnerabilities in software packages are a significant concern for developers and users alike. Patching these vulnerabilities in a timely manner is crucial to restoring the integrity and security of software systems. However, previous work has shown that vulnerability reports often lack proof-of-concept (PoC) exploits, which are essential for fixing the vulnerability, testing patches, and avoiding regressions. Creating a PoC exploit is challenging because vulnerability reports are informal and often incomplete, and because it requires a detailed understanding of how inputs passed to potentially vulnerable APIs may reach security-relevant sinks. In this paper, we present PoCGen, a novel approach to autonomously generate and validate PoC exploits for vulnerabilities in npm packages. The approach combines the complementary strengths of LLMs (e.g., understanding informal vulnerability reports), static analysis (e.g., identifying taint paths), and dynamic analysis (e.g., validating generated exploits). PoCGen successfully generates exploits for 71% of the vulnerabilities in the SecBench.js dataset. This success rate significantly outperforms a recent baseline (by 38 absolute percentage points), while imposing an average cost of only $0.02 per generated exploit. Moreover, PoCGen generates successful exploits for 60% of 126 recent real-world vulnerabilities, which helped augment five recent vulnerability reports in the GitHub Security Advisories database with PoCGen-generated PoC exploits.

CCS Concepts: • **Security and privacy → Software and application security**.

Additional Key Words and Phrases: Vulnerability, Exploit Generation, Large Language Models

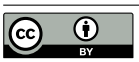
ACM Reference Format:
Deniz Simsek, Aryaz Eghbali, and Michael Pradel. 2026. PoCGen: Generating Proof-of-Concept Exploits for Vulnerabilities in Npm Packages. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE171 (July 2026), 22 pages. https://doi.org/10.1145/3808178

1 Introduction

Security vulnerabilities pose a major threat to software and users alike, with the number of reported vulnerabilities increasing each year. In 2024 alone, over 40,000 CVEs were disclosed, an increase of 38% over the previous year [1]. As software ecosystems become more complex and interdependent, mitigating vulnerabilities becomes increasingly challenging. This holds particularly for Node.js and its package manager, npm, which form the backbone of the JavaScript and TypeScript ecosystem. With millions of packages and a dense network of dependencies, the npm ecosystem is susceptible to a wide range of security risks [52], including transitive vulnerabilities, where a single vulnerable package can propagate security risks across thousands of applications [45].

When a vulnerability is discovered, it is typically reported to the developers of the affected package, who are then expected to create a patch to fix the issue. Once the vulnerable software is fixed,

Authors' Contact Information: Deniz Simsek, University of Stuttgart, Stuttgart, Germany, dsimk@pm.me; Aryaz Eghbali, CISPA Helmholtz Center for Information Security, Stuttgart, Germany, aryaz.egh@gmail.com; Michael Pradel, CISPA Helmholtz Center for Information Security, Stuttgart, Germany, michael@binaervarianz.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE171
https://doi.org/10.1145/3808178

```
An issue in parse-uri v1.0.9 allows attackers to cause a Regular expression Denial of Service
(ReDoS) via a crafted URL.
```

Fig. 1. CVE-2024-36751 report with no PoC exploit in the report.

or some time has passed since the initial vulnerability report, the vulnerability report is published as a Common Vulnerabilities and Exposures (CVE) entry. The process of fixing vulnerabilities is often facilitated by a proof-of-concept (PoC) exploit, which demonstrates how the vulnerability can be exploited in practice. Moreover, PoC exploits are useful for testing the patch and preventing regressions in the future. However, many vulnerability reports lack a PoC exploit [18], and even many CVE reports do not have any PoC exploits. For example, in the SecBench.js [3] dataset, only 179 out of 560 CVEs contain a PoC exploit in the report. Furthermore, the publicly available exploits are not reliable, and in some cases malicious themselves [43].

As a real-world example, consider the vulnerability CVE-2024-36751 in the "parse-uri" package shown in Fig. 1. The vulnerability report informally describes the problem, but it does not contain an executable code example to reproduce it, i.e., there is no PoC exploit. The process of creating PoC exploits is often time-consuming and requires a deep understanding of the codebase, the vulnerability, and the underlying technology [3]. Particularly in the case of Fig. 1, the vulnerability description does not mention which function is vulnerable, and does not provide any information about the input that triggers the vulnerability.

One way to address the challenge of generating a PoC exploit that is missing in a given vulnerability report is to leverage the capabilities of large language models (LLMs). LLMs have demonstrated their effectiveness in various software engineering tasks, including code completion [12], test generation [24, 34], program repair [5, 19, 20, 41, 47], and repository setup [6]. They are especially good candidates because of their ability to understand natural language and other informal, imprecise, and vague information provided in vulnerability reports. Moreover, with their understanding of different vulnerability types, LLMs can generate payloads for exploits in a targeted manner. However, LLMs alone may not be sufficient to generate successful PoC exploits, as they lack the necessary context about the codebase, such as implementation details of the vulnerable code, and have only a limited ability to reason about the behavior of the code.

Another way to generate PoC exploits is to deploy traditional testing and analysis techniques, such as fuzzing [8] and symbolic execution [30]. The currently most effective such technique is a recent approach called *Explode.js* [30], which uses taint analysis, a set of exploit templates, and symbolic execution to produce exploits that trigger vulnerabilities in npm packages. Explode.js has been shown to generate PoC exploits for 182 out of 560 vulnerabilities (32%) in the SecBench.js dataset, which is helpful, but still leaves many vulnerabilities without a PoC exploit. We attribute this gap to the fact that Explode.js uses only traditional program testing and analysis techniques, leveraging neither the information provided in vulnerability reports nor the capabilities of LLMs.

This paper addresses these limitations by presenting PoCGen, a novel approach for generating PoC exploits from informal vulnerability reports.

PoCGen takes as input an informal description of a vulnerability, as found in CVE reports, as well as the vulnerable code base, and automatically generates an executable JavaScript file that demonstrates how to exploit the vulnerability. The approach consists of four iteratively executed components: (i) understanding the vulnerability and extracting source-level information, (ii) generating a candidate exploit, (iii) validating the exploit, and if necessary, (iv) refining the prompt to obtain an improved exploit. These components use a combination of LLM prompting and static and dynamic analysis techniques: Component (i) uses dynamic analysis to explore the

```
async function exploit() {
    const parseuri = require("parse-uri");
    // This input is designed to cause excessive backtracking in the regex
    const craftedInput = 'http://example.com/' + 'a'.repeat(30000) + '?key=value';
    const result = await parseuri(craftedInput);
}
await exploit();
```

Fig. 2. PoCGen-generated PoC exploit for CVE-2024-36751.

package's exported functions, prompts the LLM with the given vulnerability report to identify potentially vulnerable functions, and queries a static taint analysis to extract taint paths. Component (ii) generates a candidate exploit using an LLM. Component (iii) executes and validates the candidate exploit against a dynamic analysis-based test oracle. If the candidate exploit is not valid, PoCGen uses component (iv) and refines the prompt using a set of refiners that provide static or dynamic information to component (ii) where the LLM attempts again to generate a valid exploit. This process continues until either a valid exploit is generated or until exceeding the computational budget.

To generate a PoC exploit for the ReDoS vulnerability in Fig. 1, the main challenge is to construct a payload that, when passed to the vulnerable function, triggers a security-relevant action. The goal of ReDoS is to exploit the regular expression engine's backtracking behavior, leading to excessive resource consumption. After analyzing the codebase for potential vulnerable functions, PoCGen identifies the package's default exported function as the likely vulnerable entry point, and extracts usage examples from the codebase. In its first attempt, PoCGen generates an exploit that passes a crafted input string designed to trigger the ReDoS vulnerability. However, this payload is invalid as it does not cause the expected backtracking behavior. Once PoCGen executes the initial candidate exploit, it determines that the exploit fails and, reasoning about the vulnerability, the code, and the runtime information, generates a new payload. After multiple refinements, PoCGen generates a new exploit that triggers the ReDoS vulnerability, as shown in Fig. 2.

We evaluate PoCGen on SecBench.js, a benchmark of vulnerable npm packages with path traversal, prototype pollution, command injection, code injection, and ReDoS vulnerabilities. Our results show that PoCGen successfully generates exploits for 395 out of 560 vulnerabilities (71%), outperforming the previous state of the art [30] by 38 absolute percentage points. When measuring the costs due to LLM usage, we find that PoCGen incurs an average cost of only $0.02 per vulnerability, which is reasonably low for adoption in practice. To further validate the practical usefulness of PoCGen, we use it to generate PoC exploits for 126 recent real-world vulnerabilities, where PoCGen successfully generates PoC exploits for 76 of them (60.3%). Five of the generated exploits have been added to the vulnerability reports on the GitHub Security Advisories database.

We envision the approach to be useful for developers and security researchers. By using PoCGen, developers of npm packages can generate PoC exploits for vulnerability reports they receive to help them understand the vulnerability and how to address it. They can also use the PoC exploits to test their patches and even add them to their test suites as regression tests. Moreover, security researchers can automate reporting vulnerabilities to downstream packages, by automating the PoC exploit generation process.

In summary, this paper makes the following contributions:

- A novel approach to autonomously generate and validate PoC exploits for vulnerabilities in npm packages by combining LLMs with static and dynamic analysis.

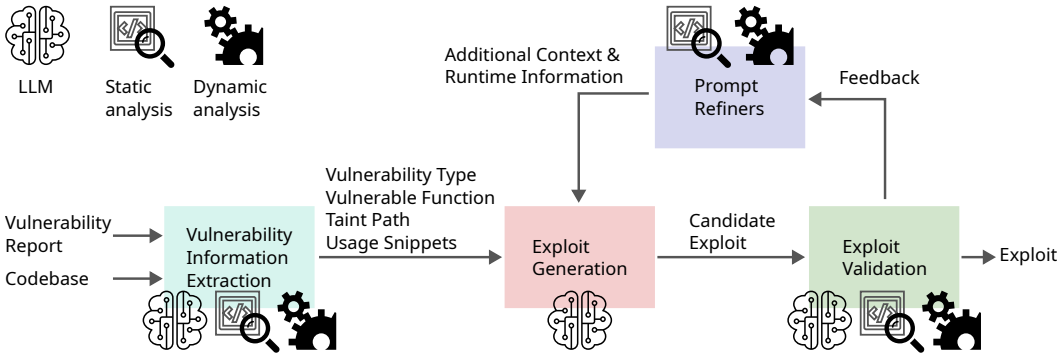


Fig. 3. Overview of PoCGen.

- Empirical evidence of the effectiveness of PoCGen on 560 real-world vulnerabilities, covering five common vulnerability types, as well as its utility for augmenting vulnerability reports that were previously missing PoC exploits.
- We share our code and data to foster future research (Section 7).

2 Approach

2.1 Overview

Figure 3 provides an overview of PoCGen. The approach consists of four main components, which combine LLM prompting, static analysis, and dynamic analysis. PoCGen takes as input a vulnerability report, which is an informal description of the vulnerability, and the codebase of the vulnerable package. Vulnerability reports originate from several sources, such as vulnerability databases (e.g., the CVE database, GitHub Security Advisories, and Snyk), bug and issue trackers, or security mailing lists.

The first step of PoCGen (Section 2.2) is to extract information about the vulnerability and the codebase. This step is necessary because the natural language description in a vulnerability report is often vague and does not provide enough information to directly generate a PoC exploit. The approach extracts the vulnerability type, the likely vulnerable function, taint paths to vulnerable sinks, and usage examples.

In the second step (Section 2.3), PoCGen uses the extracted information to generate a candidate exploit. To this end, the approach compiles all the available information into a prompt that asks the LLM to generate a PoC exploit.

Once the LLM generates a candidate exploit, the third step of PoCGen (Section 2.4) executes and validates the candidate exploit. The approach uses multiple validation mechanisms, including a set of runtime checkers that are specific to each vulnerability type. For example, for command injection vulnerabilities, the validation checks whether a specific command is executed after running the candidate exploit, while for prototype pollution vulnerabilities, it checks whether a specific property is added to the global object.

If the candidate exploit is successfully validated, PoCGen returns it to the user and the approach terminates. Otherwise, the fourth step of the approach (Section 2.5) refines the exploit generation prompt by adding additional context based on static analysis and the runtime information obtained during exploit generation. This process continues until either finding a valid exploit or exhausting the computational budget.

2.2 Vulnerability Information Extraction

In this component, PoCGen extracts four pieces of information from the vulnerability report and the codebase to provide as context to the exploit generation component.

2.2.1 Vulnerability Type. First, PoCGen identifies the type of vulnerability. This is crucial to our approach as the type of vulnerability determines the goal of the exploit and how it should be validated. To do this, we prompt the LLM with the vulnerability report and ask it to identify the type of vulnerability. As the vulnerability report is written in natural language, and the description is informal, an LLM is a suitable tool to extract information from it. We provide the LLM with the five vulnerability types supported by our approach: path traversal, prototype pollution, command injection, code injection, and Regular Expression Denial of Service (ReDoS). PoCGen prompts the LLM to select the most relevant vulnerability type from the list.

In addition to the vulnerability type, the LLM also determines if the vulnerability is exploitable remotely, such as a vulnerability in an http server that requires triggering the vulnerability via a request, or locally, where the vulnerability is triggered by importing the package and calling a function.

2.2.2 Vulnerable Function. In our approach and throughout this paper, we refer to the function that is accessible to an attacker and is the entry point for the exploit as the *vulnerable function*. Finding the vulnerable function is helpful for generating the PoC exploit, as it provides information about the input types and possible values, and potentially the vulnerable execution path of the function. To identify the vulnerable function, we first load the package and dynamically extract all the functions that it exports. We then prompt the LLM with the vulnerability report and ask it to identify the vulnerable function from the extracted functions. Since the vulnerability report can be vague, and multiple functions can be candidates for the vulnerable function, we prompt the LLM to rank the functions based on their likelihood of being the vulnerable function. This ranked list of candidate functions is the input to the next step which is to extract taint paths.

2.2.3 Taint Path Extraction. To generate a successful PoC exploit, it is crucial to understand whether and how the input to the vulnerable function flows through the code and reaches the sensitive operations, commonly referred to as vulnerable sinks. Taint analysis is a technique that tracks the flow of data through the code by marking a certain input as tainted and propagating this taint through the code. Moreover, to narrow down the search space for the vulnerable function, PoCGen attempts to extract a taint path from each of the candidate functions identified in the previous step to a vulnerable sink for locally exploitable vulnerabilities. If no taint path is found for a candidate function, that function is not considered for exploit generation. For remotely exploitable vulnerabilities, we also run the taint analysis starting at remote access endpoints, such as http request handlers.

In our approach we define a taint path as a sequence of source code lines starting with the signature line in the definition of the vulnerable function, and ending with a vulnerable sink. Any lines of code that propagate the taint are also included in the taint path. To provide more context to the LLM, for each line in the taint path, we include three lines before and after it as additional context. If these context windows overlap, we merge them to avoid duplication.

We use the static taint analysis provided by CodeQL[1]. For each vulnerability type, we use the vulnerable sinks and taint propagation rules specified in the JavaScript security library of CodeQL[2].

PoCGen processes the ranked candidate functions in batches of 50, querying the taint analysis for each batch to extract at least one taint path; this also serves to pinpoint the vulnerable function.

[1] https://codeql.github.com/

[2] The module `semmle.javascript.security`

The taint analysis of CodeQL is designed for industry-level security analysis, which requires high precision and few false positives. This means that the taint analysis may not find all taint paths.

Hence, if the first taint tracking attempt does not find any taint paths, our approach retries the taint analysis with our own extended set of taint propagation rules and vulnerable sinks. If the extended taint analysis is also unsuccessful, PoCGen prompts the LLM to guess the vulnerable sinks, and if that also fails, it falls back to a combination of static and dynamic taint analysis. Concretely (anticipating Section 2.3), it first prompts the LLM to generate an initial exploit for the vulnerable function, then executes that exploit and runs the static taint analysis on the functions called during execution. If any of the executed functions have a taint path to a vulnerable sink, the approach uses this taint path as the taint path for the vulnerable function.

If PoCGen still does not find a taint path, it proceeds to the next batch of candidate functions from the ranking and repeats the process. If the approach exhausts all candidate functions without finding any taint paths, it considers the exploit generation attempt as failed and does not move to further steps of the approach.

The output of this step is a sequence of code snippets interleaved with natural language explanations of how the code propagates the taint, shown in Fig. 4, with multiple sections if the taint path spans multiple files. In each section, there is a header specifying the file, followed by the taint path and its additional context from that file. The taint path lines are also marked by a comment at the end of the line.

The package `djv` in Fig. 4, which is a JSON schema validator, had a code injection vulnerability (CVE-2020-28464). The vulnerable function `import` takes a JSON string as input, parses it, and then calls the `restore` function with each of the attributes of the parsed object. In the restore function, a new function is dynamically created where the body of the function comes from the parsed JSON string. As shown in Fig. 4, a malicious input (e.g., JSON string containing arbitrary JavaScript code) to the `import` function, flows through to the `restore` function, and is used as the body of a dynamically created function, which is a code injection vulnerability.

2.2.4 Usage Snippets. To help the LLM generate a valid exploit, PoCGen extracts usage examples of the vulnerable function from the codebase. These examples allow the LLM to understand the function signature, and any setup that is required to call the function. We extract usage snippets both from the source code and from the documentation of the package. The usage snippets from the source code are extracted from test files using static analysis, by finding all the call sites of the vulnerable function. For the usage snippets from the documentation, we first extract all code pieces in the documentation, wrapped in triple backticks ("```"), and then we prompt an LLM to determine if they are usage examples of the vulnerable function. If they are, we also prompt the LLM to summarize them.

2.3 Exploit Generation

PoCGen assembles a prompt for the LLM to generate the PoC exploit. Figure 6 shows the prompt template that we use for generating the exploit. The prompt starts with naming the vulnerable function (as `vulnerableFunction`) and the vulnerability type (as `vulnerabilityType`), which are extracted in the previous step, followed by a description of the vulnerability (as `vulnerabilityDescription`), which is the text of the vulnerability report. Then, we provide the example usages of the vulnerable function (as `usageSnippets`), which are extracted from the codebase in the previous step.

Next, the prompt describes what the generated exploit should look like by providing the skeleton of the exploit code (as `exploitSkeleton`), and also providing exploits of similar vulnerabilities (as `similarExploits`). The skeleton of the exploit code is a fixed template that contains the definition of the exploit function, its call, and loading the vulnerable function from the vulnerable package.

````
Vulnerable method `import` of class `Environment` located in djv/lib/djv.js:
```js
import(config) { // tainted: "config"
  const item=JSON.parse(config) // tainted: "config"
  let restoreData=item // tainted: "item"
  if (item.name && item.fn && item.schema) {
    restoreData={
      [item.name]: item,
    }
  }
  Object.keys(restoreData).forEach((key)=>{ // tainted: "restoreData"
    const {name, schema, fn: source}=restoreData[key] // tainted: "restoreData"
    const fn=restore(source, schema, this.options) // tainted: "source"
    this.resolved[name]={
      name,
      schema,
```
Call to `restore`:
```js
function restore(source, schema, {inner}={}) { // tainted: "source"
  const tpl=new Function("schema", source)(schema) // tainted: "source"
  if (!inner) {
```
````

Fig. 4. Example of a taint path extracted by PoCGen from the package djv with code injection vulnerability (CVE-2020-28464).

```
  const path = require('doc-path');
  let document = {
      Make: 'Nissan',
      Features: [ { feature: 'A/C' } ]
  };
  console.log(path.setPath(document, 'Color.Interior', 'Tan'));
  /* {
          Make: 'Nissan',
          Features: [ { feature: 'A/C' } ]
          Color: { Interior: 'Tan' }
      } */
  console.log(path.setPath(document, 'StockNumber', '34567'));
  /* {
          Make: 'Nissan',
          Features: [ { feature: 'A/C' } ]
          Color: { Interior: 'Tan' },
          StockNumber: '34567'
      } */
  console.log(path.setPath(document, 'Features.cost', '$0 (Standard)'));
  /* {
          Make: 'Nissan',
          Features: [ { feature: 'A/C', cost: '$0 (Standard)' } ]
          Color: { Interior: 'Tan' },
          StockNumber: '34567'
      } */
```

Fig. 5. Example of extracted usage snippets for the vulnerable function setPath in the doc-path package.

```
`{{vulnerableFunction}}` is vulnerable to {{vulnerabilityType}}.
## Vulnerability Description:
```
{{vulnerabilityDescription}}
```
{{> usageSnippets }}
{{> exploitSkeleton }}
## Task:
Create an exploit that uses the {{vulnerabilityType}} in `{{vulnerableFunction}}` to
{{vulnerabilityTrigger}}.
Respond with the full exploit code and explain why it works.
If there is an exception thrown, do not try to handle it and pass it on.
Enclose the exploit code in backticks and define the exploit within a function named `exploit`.
{{> similarExploits }}
## Source code:
{{taintPathSnippets}}
```

Fig. 6. Prompt template for the first round of exploit generation.

To extract similar exploits, our approach uses BM25 to find the three most similar vulnerability descriptions from the SecBench.js dataset to the current vulnerability report, and provides their exploits as examples.

After providing the context, the prompt specifies the task to the LLM, which is to create an exploit that uses the vulnerability in the vulnerable function to trigger a security-relevant action (as `vulnerabilityTrigger`). This action is specifically defined for each vulnerability type, to facilitate the validation of the generated exploit. Finally, the prompt provides the taint path snippets (as `taintPathSnippets`), which are extracted in the previous step.

2.4 Exploit Validation

For each vulnerability type, PoCGen uses a specific validation mechanism to check whether the generated exploit is valid. These mechanisms are inspired by the validation checks in SecBench.js [3]. Each validator is designed to check for an event that is commonly considered as the effect of a successful exploit for the corresponding vulnerability type. For example, for the path traversal vulnerability, if the exploit is able to access a flag file in one or more directories above the current directory (e.g., `../../flag.txt`), the exploit is considered valid. Our prompts force the model to generate exploits that trigger these specific events, which makes the validation process more straightforward and reliable. Therefore, passing the validator is a requirement for an exploit to be considered valid. This means that, by definition, we do not have any false negatives, as any exploit that does not trigger the expected event is considered to be unable to exploit the vulnerability. We explain the checks each validator performs for each vulnerability type in the following.

2.4.1 Path Traversal. The goal of path traversal exploits is to access a file in the root directory. In the validators of SecBench.js this was implemented as being able to read a flag file in the root directory. However, this would render exploits for vulnerabilities that perform other file operations invalid. To mitigate this, we consider all file system operations during runtime, and check the normalized path of the accessed file. If the normalized path is evaluated to `flag.txt` in a directory above the current directory, the exploit is considered valid.

2.4.2 Prototype Pollution. In the case of prototype pollution vulnerabilities, the goal of the exploit is to add a property named `exploited` to the global `Object.prototype` object. However, the property

might be set by a direct assignment in the exploit code rather than through the vulnerable function, which would not constitute a genuine exploit. To filter out such cases, we check whether the exploit code contains a direct assignment of the form `*[__proto__|prototype].exploited = *`. If such an assignment exists and the property is set, the exploit is considered invalid.

2.4.3 Command Injection. For command injection vulnerabilities, the goal of the exploit is to execute the command `/usr/bin/genpoc`. This command writes a file to the file system, which is then checked by the validation checker.

2.4.4 Code Injection. For code injection vulnerabilities, the goal of the exploit is to call the built-in `process.seteuid` function, with parameter 42. This indicates that the exploit was able to access the `process` module. Access to this module implies the ability to execute arbitrary code in the context of the Node.js process. To ensure that the exploit triggers the sink through the vulnerable function rather than calling it directly, we statically check whether `process.seteuid(42)` is called directly in the exploit code. If there is such a direct call, we consider the exploit invalid, as it bypasses the vulnerable function.

2.4.5 ReDoS. For ReDoS vulnerabilities, the goal of the exploit is to cause a denial of service by taking a long time to execute. We hook the string and regex functions in the V8 engine to measure the time these functions take. If the execution time of a function exceeds 1,500 milliseconds, we consider the exploit as valid.

2.4.6 Sanity Checks. Passing the vulnerability-specific validation checks is a necessary condition for an exploit to be considered valid, but it is not sufficient as it is possible to achieve the desired outcome with other means. For example, in case of ReDoS, the LLM might generate a new regular expression that takes a long time to execute, but is not related to the vulnerable function. Therefore, in addition to the vulnerability-specific validation checks, PoCGen searches the call stack when the vulnerability is triggered for a call to the vulnerable function. If no such call is found, the exploit is considered invalid.

As a last step in the validation process of any vulnerability type, we prompt the LLM to check whether the exploit actually triggers the vulnerability described in the report. This is done to filter out any invalid exploits that passed the previous validation checks.

2.5 Prompt Refinement

After the validation step, if the exploit is not valid, PoCGen refines the prompt to generate a new candidate exploit. PoCGen uses a set of refiners that provide static or dynamic information to the LLM to help it generate a valid exploit.

2.5.1 Context Refiners. The first set of refiners are the *context refiners*, which provide additional context to the LLM to help it generate a valid exploit. Since the taint path only contains the taint propagation lines, checks on taint values are not included in the taint path. Therefore, the LLM might lack information about input-validation guards in the code that could prevent a malicious input from reaching the sink. To address this, we provide a *body refiner*, which provides the full body of any function that has at least one line of code in the taint path.

However, there can be checks that happen via function calls that are not in the taint path. To address this, we also provide a *missing declaration refiner*, which provides the LLM with the ability to ask for definitions of variables and functions in the taint path, through the function calling format of OpenAI's API. The LLM can output a list of identifiers for which it needs their definitions, and PoCGen will provide the definitions of these identifiers in the prompt using the V8's API, which itself uses a combination of static and dynamic analyses.

2.5.2 Runtime Refiners. The second set of refiners are *runtime refiners*, which add information about the execution to the prompt. The refiners in this category are the *error refiner*, the *coverage refiner*, the *debugger refiner*, and the vulnerability-specific refiners.

Since the exploit generated by the LLM can have runtime errors, for example from a wrong API usage, the *error refiner* provides the LLM with the error message that was thrown during the execution of the candidate exploit.

The *coverage refiner* provides the LLM with the coverage information of the candidate exploit, as markings in comments at the end of each line in the taint path. This information is useful for the LLM to understand which parts of the code were not executed. If the vulnerable sink was not executed, the information provided by this refiner can help the LLM to generate a new exploit that reaches the vulnerable sink.

The *debugger refiner* provides the LLM with a debugger-like tool. The LLM can output a list of expressions, for which it needs the runtime values. Using the Inspector API of Node.js[3], PoCGen evaluates these expressions during the execution of the exploit. These values are provided as comments in their respective lines of the taint path in the prompt.

There are cases where the LLM generates an exploit that reaches the vulnerable sink, but the exploit fails the validation checks due to a wrong input. For path traversal, command injection, and code injection vulnerabilities, we provide specific refiners that hook into the vulnerable sinks and provide the runtime values passed to these functions. This form of feedback allows the LLM to understand how the input it generated is transformed, which can help it generate a valid exploit in the next iteration. For path traversal vulnerabilities, the refiner provides the values passed to the file system functions, like `fs.readFile` and `fs.open`. For command injection vulnerabilities, it provides the values passed to the `spawn` function. Finally, in case of code injection vulnerabilities, the refiner provides the values passed to the most common sink functions, like the `Function` constructor.

2.5.3 Combining Refiners. Based on the different refiners described above, PoCGen implements a refinement loop that iteratively improves prompts until a valid exploit is generated or the budget is exhausted. Figure 7 shows the algorithm for the refinement loop. Initially, a seed prompt is placed in the priority queue. In every refinement attempt, PoCGen chooses one refiner from the front of this queue. Each time the approach uses a refiner, it assigns a score based on the number of new errors the respective exploit causes, and the number of steps from the taint path it covers. It then adds the refined prompt to the priority queue with the score. Moreover, if a refinement generates a prompt that is already used, PoCGen does not query the LLM again, and moves to the next refiner. To keep the prompts concise, in each refinement, PoCGen removes parts of the prompt that the LLM has correctly used in the previous attempts. For example, if the exploit generated in the previous step uses the vulnerable function correctly, PoCGen removes the usage snippets from the prompt. The refinement process continues until either a valid exploit is generated, or the computational budget is exhausted.

3 Evaluation

We evaluate PoCGen on 560 vulnerabilities in npm packages to answer the following research questions:

- RQ1: How effective is PoCGen in generating PoC exploits?
- RQ2: How much does each component of PoCGen contribute to the overall effectiveness?
- RQ3: How much does it cost to generate PoC exploits in terms of money and time?
- RQ4: How effective is PoCGen in recent real-world vulnerability reports?

[3] https://nodejs.org/api/inspector.html

```
prompts ← PriorityQueue{(0, getSeedPrompt())}
usedRefiners ← {}
seenExploits ← {}
while prompts ≠ {} and |usedRefiners| < 30 do
    prompt ← top(prompts)
    prompts ← prompts \ {prompt}
    prompts ← prompts ∪ {prompts with more info}
    exploit ← generateExploit(prompt)      // via LLM
    if exploit ∉ seenExploits then
        result ← runAndValidate(exploit)
        if result.success then
            return exploit
        endif
        correctlyUsedInfo ← getCorrectlyUsedInfo(prompt, result) // Information in the prompt
        ↪  that the LLM used correctly
        prompts ← prompts ∪ {(
          result.taintSteps + result.newErrors,
          prompt + result.errors - correctlyUsedInfo
        )}
        usedRefiners ← usedRefiners ∪ {used refiners}
        seenExploits ← seenExploits ∪ {exploit}
    endif
endwhile
return failure
```

Fig. 7. Algorithm for the refinement loop.

- RQ5: What are the characteristics of vulnerabilities that affect the success of PoCGen?

3.1 Experimental Setup

3.1.1 Dataset. We use the SecBench.js dataset [3] to evaluate PoCGen The dataset contains 600 vulnerable npm packages with code injection, command injection, prototype pollution, path traversal, and ReDoS vulnerabilities. We exclude packages that have been removed from the npm registry. This leaves us with a total of 560 vulnerabilities to evaluate our approach. The vulnerable packages in SecBench.js are packages with various sizes spanning from 1 line of code to 892k lines of code, with the largest package being "jointjs". The average package has 8.3k lines of code. Moreover, the vulnerabilities in these packages are real-world vulnerabilities and the average package in this benchmark is depended on by 947.62 packages, meaning they are used in many other JavaScript projects.

3.1.2 Implementation and Configuration. We run all experiments on an Ubuntu 22.04 machine with Intel Xeon(R) Silver 4214 CPU, with 256 GB of RAM. The experimental setup uses Node.js version 22.11.0, running on a modified V8 engine that throws an error if a configurable backtracking limit is exceeded. For static taint analysis, we use CodeQL version 2.20.4.

As the LLM, we use OpenAI's `gpt-4o-mini-2024-07-18` and `gpt-5-mini-2025-08-07` models through the OpenAI API. We use the `gpt-5-mini` model with "minimal" reasoning effort as the default model for all experiments, unless otherwise specified. We use a system prompt that assigns the role of a security researcher specialized in creating exploits for the identified security class to the LLM. This is done to reduce refusals to generate exploits by the LLM for safety reasons. For each vulnerability, we allocate a time budget of one hour, a token budget of 300k input tokens, and 100k output tokens. The maximum refinement budget is set to 30 iterations.

3.1.3 Baselines. We compare PoCGen against two baselines: Explode.js [30] and an LLM-based agent using the Mini-SWE-Agent framework[4] [44]. Explode.js is a state-of-the-art approach for generating PoC exploits. It first uses a static dataflow analysis to detect which exported functions reach a vulnerable sink, which is then used to create an exploit template. Then, using symbolic execution, it generates symbolic inputs to exploit the vulnerability. Finally, it uses an SMT solver to generate concrete inputs that trigger the vulnerability.

Recently, LLM-based agents have shown great promise in software engineering tasks, such as resolving issues [44], repairing bugs [5], and executing arbitrary projects [6]. Therefore, we also implement an LLM-based agent using the Mini-SWE-Agent framework as a second baseline. Our version of the Mini-SWE-Agent is configured to target the PoC exploit generation task, and is provided with the same information as PoCGen, i.e., the vulnerability report without any PoC exploits. We limit the agent to run for at most 30 steps, similar to our approach. The Mini-SWE-Agent can use the command line (bash) to perform various tasks, such as navigating the file system, reading and writing files, and executing code.

3.2 RQ1: Effectiveness

We evaluate the effectiveness of PoCGen in generating PoC exploits for vulnerabilities in SecBench.js as done in previous work [30]. We measure the success rate of our approach and compare it to the baselines. We also report the number of failed attempts, and the number of false positives, which are exploits that pass the validator, but do not trigger the vulnerability through the vulnerable function. For each PoC exploit that PoCGen generates, one of the authors manually inspects it to determine whether it is a false positive or a successful exploit.

In our experiments, we run PoCGen three times on all 560 vulnerabilities in SecBench.js to mitigate the randomness of LLMs. We use the reported results of Explode.js[5] on the same set of vulnerabilities. However, since agentic approaches require evaluating their trajectory to ensure correctness, which is expensive and time-consuming, we limit the number of vulnerabilities evaluated with our agentic baseline approach to 100. We randomly sample 20 vulnerabilities from each of the five vulnerability types in SecBench.js, and run Mini-SWE-Agent on them.

Figure 8 shows the effectiveness of PoCGen, Mini-SWE-Agent, and Explode.js on the SecBench.js dataset. PoCGen successfully generates PoC exploits for 395 out of 560 vulnerabilities, on average, which is 71% of the vulnerabilities. The standard deviation of the number of generated exploits across the three repeated runs is 4.5, amounting to 0.8% of the dataset size. Explode.js successfully generates PoC exploits for 182 out of 560 vulnerabilities, which is 32% of the vulnerabilities. Mini-SWE-Agent generates successful PoC exploits for 28 out of the 100 vulnerabilities on average, with a standard deviation of 2.65. The results show that PoCGen outperforms Explode.js by 38 and Mini-SWE-Agent by 42 absolute percentage points.

When comparing the effectiveness of PoCGen on different vulnerability types, we find that PoCGen performs best on path traversal, and command injections, with success rates of above 79%. In case of Explode.js, it also performs better on path traversal and command injection compared to the other vulnerability types, with success rates between 50% and 60%. Since Explode.js does not support ReDoS vulnerabilities, its success rate on ReDoS vulnerabilities is 0%. Mini-SWE-Agent succeeds in 40% of cases on prototype pollution and command injection, while only succeeding in 10% of cases on path traversal vulnerabilities.

[4] https://mini-swe-agent.com
[5] https://github.com/formalsec/explode-js/blob/71ec17fe90f29e236b01b8cad02685344f8aff10/bench/explode-vulcan-secbench-results.csv

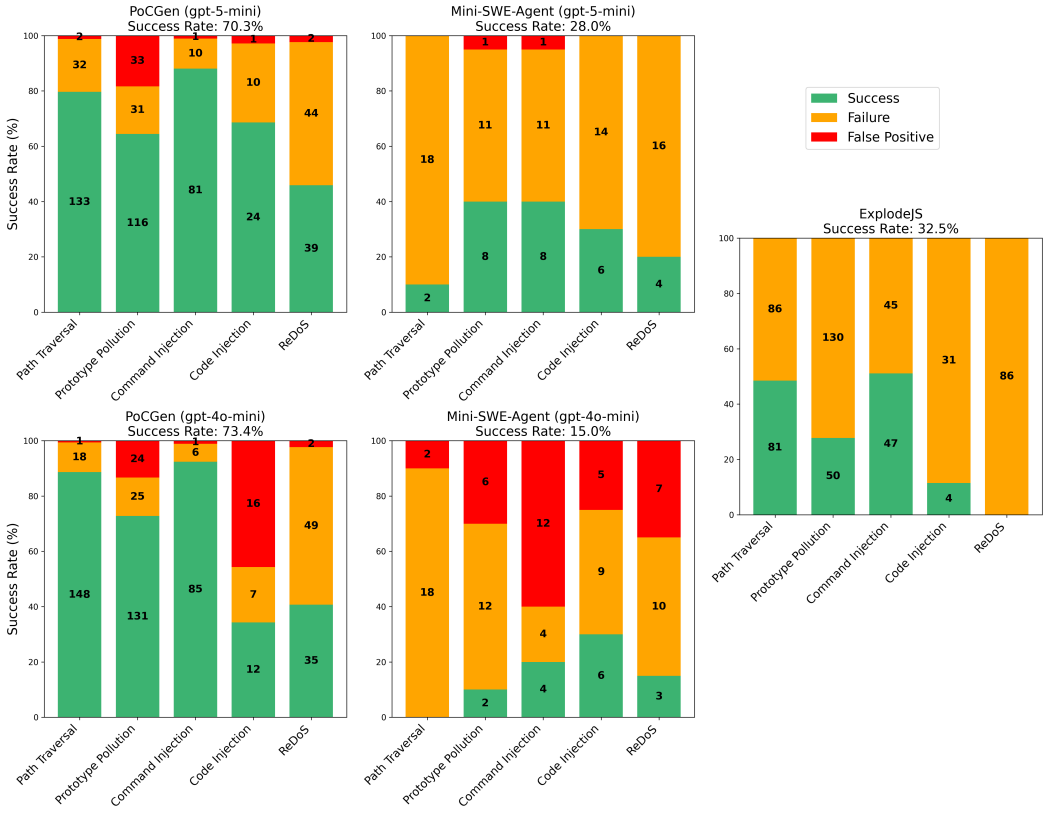


Fig. 8. Effectiveness of PoCGen, Mini-SWE-Agent, and Explode.js on SecBench.js.

We also evaluate PoCGen and Mini-SWE-Agent using the `gpt-4o-mini` model, which is an older and less capable model than `gpt-5-mini`. The results show that PoCGen maintains a high success rate, while Mini-SWE-Agent's effectiveness significantly drops by 13 percentage points, with an increase in false positives.

By investigating the reasons that PoCGen fails to generate a successful PoC exploit, we observe that for 4% of the vulnerabilities, the approach runs through 30 refinement iterations without generating a working exploit. For 3.6% of vulnerabilities, the token limits we set are reached, and the approach stops. The packages in 1.2% of the vulnerabilities do not export any functions. Finally, for 1.2% of vulnerabilities, PoCGen itself evaluates its own generated exploits as false positives.

3.3 RQ2: Ablation Study

To evaluate the impact of each component on the effectiveness of PoCGen, we perform an ablation study on the SecBench.js dataset. We evaluate the following configurations of PoCGen to measure the impact of each component on the success rate of generating PoC exploits.

- PoCGen: The complete PoCGen approach with all components.
- noTaintPath: PoCGen without the taint path analysis, described in Section 2.2.3. In this configuration, no taint analysis is done, which means the approach does not utilize taint paths to find the vulnerable source, the LLM prompt does not contain the taint path, and also

Table 1. The effect of each component of PoCGen on successfully generating PoC exploits.

Configuration	Valid Exploits	Success Rate
PoCGen	**395**	**71%**
noTaintPath	115	21%
noUsageSnippet	196	35%
noFewShot	375	67%
noDebugger	377	67%
noErrorRefiner	376	67%

both of the context refiners, i.e., the body refiner and the missing declaration refiner, that depend on the taint path are not used.
- noFewShot: PoCGen without the few-shot examples of exploits of similar vulnerabilities (Section 2.3).
- noUsageSnippet: PoCGen without the usage snippet examples (Section 2.2.4).
- noDebuggerRefiner: PoCGen without the debugger refiner (Section 2.5.2).
- noErrorRefiner: PoCGen without the error refiner (Section 2.5.2).

The success rates of these configurations on SecBench.js are shown in Table 1. These results show that all components contribute to PoCGen's overall effectiveness, with the taint analysis having the highest impact, followed by the usage snippet examples.

To better understand the contribution of iteratively refining the prompts, we also measure the number of refinement attempts required to generate a valid PoC exploit. Iterative attempts for generating PoC exploits have a significant impact on the effectiveness of PoCGen. Only 27% of the successful PoC exploits are generated in the first attempt, and 47% are generated within two to ten refinements. On average, successful PoC exploits are generated after 4 rounds of refinement.

We also analyze the component responsible for taint analysis, which has the highest impact on the effectiveness of PoCGen. The default CodeQL taint analysis is invoked for 72% of the vulnerabilities, running 1.3 times per vulnerability on average. In 39% of these cases, the default taint analysis fails, and the approach falls back to our extended taint analysis. Out of the extended taint analysis runs, 60% still fail, which leads the approach to fallback to using the LLM to guess the vulnerable sinks, which still fails in 3% of those cases. The CodeQL taint analysis for remote sources, such as http request handlers, runs for 34% of the vulnerabilities.

3.4 RQ3: Costs

To evaluate the costs of PoCGen, we measure the time and token usage of the approach. On average, each PoC exploit generation attempt takes 11 minutes. Since successful attempts stop earlier than failing attempts, which require running the refiners, the successful runs complete on average in 7 minutes. A significant portion of this time is spent on the CodeQL analyses, which take 51% of the execution time. The second-largest portion of the overall time is spent on querying the LLM, which takes 15% of the overall time, on average.

We measure the number of tokens sent to and received from the LLM. On average, for each attempt to generate an exploit, PoCGen sends 32,059 tokens to the LLM and receives 8,644 tokens in response. With the current pricing of the OpenAI API (as of February 2026), this costs $0.025 per exploit generation attempt. For the successful attempts, this cost is even lower, with only $0.009 per vulnerability, on average.

3.5 RQ4: Recent Real-World Vulnerability Reports

To further evaluate the usefulness and generalizability of PoCGen, we apply it to recent vulnerabilities reported on the GitHub Security Advisories database. We select vulnerabilities that match one of our five vulnerability types and are reported between January 1, 2025, and January 8, 2026. This results in a total of 183 vulnerability reports. We then manually inspect the vulnerability reports and filter out vulnerabilities that are withdrawn, where the vulnerable package is no longer available on npm, where the vulnerable code is in a language other than JavaScript, or where exploiting the vulnerability requires a different operating system, setting up a specific server, or some other complex environment. The final set contains 126 vulnerabilities, to which we apply PoCGen to generate PoC exploits. Since the knowledge cutoff date of `gpt-5-mini` is May 31, 2024, these vulnerabilities are not expected to be in the training data of the LLM, which allows us to evaluate the generalizability of PoCGen to new vulnerabilities.

PoCGen generates valid PoC exploits for 76 (60.3%) of these vulnerability reports. For six of the generated PoC exploits, the original vulnerability report did not have a PoC exploit. For each of them, we create a pull request to add the generated PoC exploit to the vulnerability reports. Five of the six pull requests have been accepted, and the generated exploits have been added to the corresponding vulnerability reports. The remaining pull request is still under review.

For example, the popular package `tar-fs`, with over 25 million weekly downloads, had a path traversal vulnerability (CVE-2024-12905) reported on March 27, 2025. This vulnerability affected three version ranges of this package, and before running PoCGen, there were no public PoC exploits for this vulnerability. PoCGen generates a valid PoC exploit for this vulnerability, which is shown in Fig. 9. To generate this exploit, PoCGen uses eight refinement attempts, which include the error refiner and the debugger refiner.

Another example is our motivating example shown in Fig. 1. This vulnerability, reported on January 16, 2025 as `CVE-2024-36751`, is a regular expression denial of service (ReDoS) vulnerability in the `parse-uri` package. As shown in Fig. 2, PoCGen successfully generates a PoC exploit, which is now part of the vulnerability report on the GitHub Security Advisories database.

3.6 RQ5: Qualitative Analysis

To better understand the strengths and limitations of PoCGen, we present examples of successful and unsuccessful attempts by PoCGen to generate PoC exploits.

3.6.1 Successful Example: CVE-2023-26127. The vulnerability report `CVE-2023-26127` describes a command injection vulnerability in the `n158` package. This vulnerability is due to not sanitizing the input, which then flows to the `exec` function of `shelljs`. However, the input is first passed to a copy function, as shown in Fig. 10. This means that if the `seedPath` is set to a value where the corresponding path does not exist, the exploit fails before reaching the vulnerable sink. This constraint is not easy to capture with symbolic execution, as attempted by Explode.js [30], but using execution feedback and an LLM, PoCGen is able to generate a working exploit.

In the first round of exploit generation, PoCGen generates an exploit that calls `initProject("someSeedPath", "targetPath; /usr/bin/genpoc")`. After the execution of this exploit, the runtime feedback shows that there was a runtime error happening at the `fsExtra.copySync` function, which is due to the fact that no directory called `"someSeedPath"` exists. The LLM then naively tries to fix this by changing `"someSeedPath"` to `"/path/to/valid/seed"`, which results in the same error. It takes multiple refinements until the LLM requests the definition of `shelljs`, after which the LLM reasons that it needs to create the directory for a successful exploit. This results in the final exploit shown in Fig. 11.

```javascript
const tarfs = require("tar-fs");
const fs = require("fs");
const path = require("path");

async function exploit() {
    // Create a writable stream to extract the tar content
    const extractStream = tarfs.extract('/', {
        // We can ignore the file type checks to allow the extraction of the malicious file
        ignore: (name) => false,
    });

    // Create a tar stream
    const tarStream = tarfs.pack().on('error', (err) => {
        throw err;
    });

    // Append the malicious entry to the tar stream
    tarStream.entry({ name: '/flag.txt', mode: 0o644 }, Buffer.from('This is a flag!'));

    // Finalize the tar stream
    tarStream.finalize();

    // Pipe the tar stream into the extract stream
    tarStream.pipe(extractStream);
}

// Call the exploit function
exploit();
```

Fig. 9. PoC exploit generated by PoCGen for CVE-2024-12905 in the `tar-fs` package.

```javascript
module.exports = function (seedPath, targetPath) {
  fsExtra.copySync(seedPath, targetPath);
  shell.exec('sh -c \'cd '+targetPath+' && npm install\'');
  shell.exec('sh -c \'cd '+targetPath+' && npm start\'');
}
```

Fig. 10. Example of a command injection vulnerability in the `n158` package.

```javascript
async function exploit() {
  const initProject = require("n158/src/cli/initProject");
  const seedPath = "/path/to/valid/seed";
  const targetPath = "targetPath; /usr/bin/genpoc";
  const fs = require('fs');
  if (!fs.existsSync(seedPath)) {
      fs.mkdirSync(seedPath, { recursive: true });
  }
  const result = await initProject(seedPath, targetPath);
}
await exploit();
```

Fig. 11. Example of a successful PoC exploit generation by PoCGen.

```
module.exports = function (file, cb) {
  var dirname = path.dirname(file);
  var filename = path.basename(file);
  exec('git blame ' + filename, {cwd: dirname}, ... );
}
```

Fig. 12. Example of a command injection vulnerability in the `gitblame` package.

3.6.2 Unsuccessful Example: GHSA-3486-rvxc-hrrj. The vulnerability report `GHSA-3486-rvxc-hrrj` describes a command injection vulnerability in the `gitblame` package. The package exports one function, which takes a file path as an argument and after some processing passes it to `exec`. The relevant parts of the source code are shown in Fig. 12.

PoCGen is not able to generate a working exploit for this vulnerability. The validator requires the command `/usr/bin/genpoc` to be executed, meaning the injected payload must contain a `/` character. However, because the input is split by `path.basename` and `path.dirname` before reaching `exec`, any `/` in the payload is stripped, so no payload can simultaneously satisfy both the vulnerability trigger and the validator. As a result, even with multiple refinements, PoCGen is unable to generate a working exploit, although the exploit for this vulnerability in SecBench.js uses the simple payload `"& touch gitblame"`, which avoids this constraint entirely.

4 Threats to Validity

One threat to internal validity is the LLM's potential to recall exploits from its training data. To mitigate this, we use the same LLM for the Mini-SWE-Agent baseline, which we would expect to perform similarly well as PoCGen if the results were simply due to memorization. Another threat is that the labeling process of false positive PoC exploits is done manually, which can introduce human bias. To reduce the effects of this bias, we devise a set of objective criteria that precisely define what constitutes a false positive, reducing reliance on subjective judgment. For example, prototype pollution cases where the `Object.prototype` is part of an assignment or a function call are considered false positives. A threat to external validity is that our evaluation is limited to five vulnerability types. However, these vulnerability types correspond to the five most common threat classes in server-side JavaScript [3]. Adapting the approach to other vulnerability types would require adapting our prompts, the sinks used by the taint analysis, and the validators. Another threat to external validity is that we evaluate our approach primarily on the SecBench.js dataset, which may not be representative of all vulnerabilities in the npm ecosystem. To mitigate this threat and the internal validity threat of model memorization, we also evaluate PoCGen on 126 vulnerabilities in the npm ecosystem that were reported since January 2025, i.e., after the knowledge cutoff of the LLMs we use. Our approach is implemented for JavaScript and the npm ecosystem, which may limit its applicability to other programming languages and ecosystems. We believe that the core ideas of PoCGen, i.e., how to combine LLMs with program analysis techniques, could be adapted to other contexts, but leave this for future work.

5 Related Work

Vulnerability Detection. Greybox fuzzing [4, 46], applied to source code [36] or binaries [14], is a common approach for vulnerability detection. To evaluate fuzzing, techniques for reverting fixes [50] and benchmarking methodologies [22, 25] have been proposed. Learning-based vulnerability detection includes neural classification [10, 15, 27], graph neural networks [26, 39], and combinations of LLMs with static analysis. Similar to our work, the latter leverages CodeQL to identify taint

flows [28]. To support learning-based detection, datasets from commit histories [51] and large-scale vulnerability generation approaches [31, 32] have been introduced. Vulnerability detection is orthogonal to our work, as we assume vulnerabilities are already described in a report but lack an exploit.

Detecting and Exploiting Node.js Vulnerabilities. The closest work to PoCGen is Explode.js [30], which finds vulnerabilities and generates PoC exploits for npm packages. Explode.js uses static dataflow analysis to extract the sequence of function calls required to propagate attacker input to a vulnerable sink. It then applies symbolic execution and constraint solving to generate a PoC exploit. However, Explode.js does not model external functions and libraries during symbolic execution, which limits its effectiveness, as the npm ecosystem heavily relies on small, reusable packages. PoCGen addresses this limitation by leveraging LLMs to generate exploits and by incorporating runtime feedback. LLMs, trained on large code corpora, can better predict the behavior of external functions and reason about inputs that exercise specific program paths.

Other approaches have used symbolic execution to generate exploits for vulnerabilities. FAST [21] applies bi-directional dataflow analysis to detect taint paths efficiently, enabling scalable vulnerability detection. It generates exploits by concretizing symbolic path constraints once a vulnerability is found. Node-Medic [9] and Node-Medic-FINE [8] combine dynamic taint analysis with symbolic execution to detect and generate exploits for Node.js packages. Node-Medic-FINE further incorporates fuzzing to generate inputs and explore additional execution paths. All three approaches are outperformed by Explode.js [30], which we therefore use as a baseline in our evaluation.

Test Generation for Security Vulnerabilities. Zhang et al. [48] and Gao et al. [16] use LLMs to generate unit tests for Java vulnerabilities given a PoC exploit. Their goal is to encourage developers to update vulnerable dependencies and prevent supply chain attacks. In contrast, our work generates code that directly exploits a vulnerable package, rather than exploiting it through a third-party dependency.

LLM-Assisted Attacks. Recent work has explored the potential of LLMs for attacking vulnerable software. PwnGPT [33] uses LLMs with reasoning models to generate exploits for capture-the-flag (CTF) challenges. PentestGPT [13] uses LLMs for penetration testing. Both of these approaches focus on system-level penetration tasks, which differs from the more isolated task of providing a PoC exploit for an individual package. Moreover, these approaches do not enable the LLM to directly benefit from sophisticated tools, e.g., for statically analyzing code or interactively debugging a candidate exploit. Xu et al. [42] developed an LLM agent with command-line access to exploit vulnerabilities in Linux and Windows applications. Charan et al. [11] investigated using LLMs to generate payloads for exploiting vulnerabilities.

Vulnerability Mitigation and Repair. Mitigating vulnerabilities can involve removing unused dependencies [23] or reducing the privileges of vulnerable code [40]. Repairing vulnerabilities can be achieved by fine-tuning LLMs to find fixes [2], using LLM agents [49], or applying generative adversarial networks (GANs) [17].

JavaScript and Npm Ecosystem Security. The npm ecosystem faces various security issues, such as injection attacks [38], ReDoS [37], and malicious packages [35]. Several empirical studies have analyzed npm from a security perspective, including vulnerability propagation [29, 52] and how developers address vulnerabilities [45]. Householder et al. [18] found that most vulnerability reports lack a public PoC exploit for at least one year. Yadmani et al. [43] further showed that many PoC exploits on GitHub are themselves malicious. These findings motivate our work on automated

PoC exploit generation. To support further research, Bhuiyan et al. [3] introduced the SecBench.js dataset, and Brito et al. [7] created the VulcaN dataset.

6 Ethical Considerations

Automating the generation of exploits raises ethical concerns, as it could potentially be misused by malicious actors to create exploits for vulnerabilities that have not yet been patched. Since our approach targets "proof-of-concept" exploits, which are typically used for testing and demonstration purposes, the generated exploits may not be directly usable for malicious purposes. Moreover, we believe that our approach can benefit the developers of vulnerable packages and the security community in general by automating part of the vulnerability management process, which can lead to faster patching and improved security.

Our evaluations have all been on public vulnerability reports, which have already been responsibly disclosed, and for which exploits may already exist. Furthermore, as our datasets consist of vulnerable packages, we were able to run all the exploits on isolated environments without risking harm to real users. Finally, we have made the source code of PoCGen and the datasets used in our evaluation publicly available to encourage further research in this area.

7 Conclusion

This paper presents PoCGen, an LLM-based approach to automatically generate proof-of-concept exploits for vulnerabilities in npm packages. PoCGen extracts information from the vulnerability report and the codebase to generate a PoC exploit using an LLM. The generated PoC exploits are validated using a set of runtime checkers, and the prompt is refined using static and dynamic information to generate a valid exploit. We evaluated PoCGen on a dataset of 560 vulnerabilities in npm packages. PoCGen generates PoC exploits for 71% of these vulnerabilities, outperforming the state-of-the-art approaches Explode.js and Mini-SWE-Agent by 38 and 42 absolute percentage points, respectively. PoCGen also generates PoC exploits for 60% of recently reported vulnerabilities from the GitHub Security Advisories database.

By automating the generation of PoC exploits, PoCGen enables developers and security teams to more rapidly understand and address vulnerabilities, reducing the time between vulnerability disclosure and patch deployment. This automation also improves regression testing, as the generated PoC exploits can be directly used to verify the effectiveness of fixes and to prevent vulnerabilities from reappearing in the future. For security researchers, PoCGen provides an automated way to evaluate the effectiveness of existing mitigation strategies across large sets of vulnerabilities. Furthermore, PoCGen can improve the quality of existing vulnerability reports, including those that are poorly documented or lack existing exploits. Finally, automated PoC generation can facilitate responsible vulnerability disclosure by providing clear, actionable evidence to affected parties, encouraging timely remediation.

Data Availability

The source code of PoCGen, the new dataset, and all experiment scripts are available at https://github.com/sola-st/PoCGen, and also as a permanent snapshot at https://doi.org/10.5281/zenodo.19550952.

Acknowledgments

This work was supported by the German Research Foundation (DFG; projects 492507603, 516334526, and 526259073).

References

[1] [n. d.]. CVE: Common Vulnerabilities and Exposures. https://www.cve.org/about/Metrics. Accessed: 2025-05-14.

[2] Berkay Berabi, Alexey Gronskiy, Veselin Raychev, Gishor Sivanrupan, Victor Chibotaru, and Martin T. Vechev. 2024. DeepCode AI Fix: Fixing Security Vulnerabilities with Large Language Models. *CoRR* abs/2402.13291 (2024). https://doi.org/10.48550/ARXIV.2402.13291 arXiv:2402.13291

[3] Masudul Hasan Masud Bhuiyan, Adithya Srinivas Parthasarathy, Nikos Vasilakis, Michael Pradel, and Cristian-Alexandru Staicu. 2023. SecBench.Js: An Executable Security Benchmark Suite for Server-Side JavaScript. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1059–1070. https://doi.org/10.1109/ICSE48619.2023.00096

[4] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2019. Coverage-Based Greybox Fuzzing as Markov Chain. *IEEE Trans. Software Eng.* 45, 5 (2019), 489–506. https://doi.org/10.1109/TSE.2017.2785841

[5] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *International Conference on Software Engineering (ICSE)*. https://doi.org/10.1109/ICSE55347.2025.00157

[6] Islem Bouzenia and Michael Pradel. 2025. You Name It, I Run It: An LLM Agent to Execute Tests of Arbitrary Projects. In *ISSTA*. https://doi.org/10.1145/3728922

[7] Tiago Brito, Mafalda Ferreira, Miguel Monteiro, Pedro Lopes, Miguel Barros, José Fragoso Santos, and Nuno Santos. 2023. Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.Js Packages. *IEEE Transactions on Reliability* 72, 4 (Dec. 2023), 1324–1339. https://doi.org/10.1109/TR.2023.3286301

[8] Darion Cassel, Nuno Sabino, Min-Chien Hsu, Ruben Martins, and Limin Jia. 2025. NodeMedic-FINE: Automatic Detection and Exploit Synthesis for Node.Js Vulnerabilities. In *Proceedings 2025 Network and Distributed System Security Symposium*. Internet Society, San Diego, CA, USA. https://doi.org/10.14722/ndss.2025.241636

[9] Darion Cassel, Wai Tuck Wong, and Limin Jia. 2023. NodeMedic: End-to-End Analysis of Node.Js Vulnerabilities with Provenance Graphs. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. 1101–1127. https://doi.org/10.1109/EuroSP57164.2023.00068

[10] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Trans. Software Eng.* 48, 9 (2022), 3280–3296. https://doi.org/10.1109/TSE.2021.3087402

[11] P. V. Sai Charan, Hrushikesh Chunduri, P. Mohan Anand, and Sandeep K. Shukla. 2023. From Text to MITRE Techniques: Exploring the Malicious Use of Large Language Models for Generating Cyber Attack Payloads. https://doi.org/10.48550/arXiv.2305.15336 arXiv:2305.15336 [cs]

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 https://arxiv.org/abs/2107.03374

[13] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. {PentestGPT}: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 847–864.

[14] Sushant Dinesh, Nathan Burow, Dongyan Xu, and Mathias Payer. 2020. RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 1497–1511. https://doi.org/10.1109/SP40000.2020.00009

[15] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *19th IEEE/ACM International Conference on Mining Software Repositories, MSR*. ACM, 608–620. https://doi.org/10.1145/3524842.3528452

[16] Yi Gao, Xing Hu, Zirui Chen, and Xiaohu Yang. 2025. Vulnerability-Triggering Test Case Generation from Third-Party Libraries. https://doi.org/10.48550/arXiv.2409.16701 arXiv:2409.16701 [cs]

[17] Jacob Harer, Onur Ozdemir, Tomo Lazovich, Christopher P. Reale, Rebecca L. Russell, Louis Y. Kim, and Sang Peter Chin. 2018. Learning to Repair Software Vulnerabilities with Generative Adversarial Networks. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3-8 December 2018, Montréal, Canada*. 7944–7954. http://papers.nips.cc/paper/8018-learning-to-repair-software-vulnerabilities-with-generative-adversarial-networks

[18] Allen D Householder, Jeff Chrabaszcz, Trent Novelly, and David Warren. 2020. Historical Analysis of Exploit Availability Timelines. (2020).

[19] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *ICSE*. 1430–1442. https://doi.org/10.1109/ICSE48619.2023.00125

[20] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, San Francisco CA USA, 1646–1656. https://doi.org/10.1145/3611643.3613892

[21] Mingqing Kang, Yichao Xu, Song Li, Rigel Gjomemo, Jianwei Hou, V. N. Venkatakrishnan, and Yinzhi Cao. 2023. Scaling JavaScript Abstract Interpretation to Detect and Exploit Node.Js Taint-style Vulnerability. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1059–1076. https://doi.org/10.1109/SP46215.2023.10179352

[22] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 2123–2138. https://doi.org/10.1145/3243734.3243804

[23] Igibek Koishybayev and Alexandros Kapravelos. 2020. Mininode: Reducing the Attack Surface of Node.js Applications. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*. USENIX Association, San Sebastian, 121–134. https://www.usenix.org/conference/raid2020/presentation/koishybayev

[24] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering, ser. ICSE*. https://doi.org/10.1109/ICSE48619.2023.00085

[25] Yuwei Li, Shouling Ji, Yuan Chen, Sizhuang Liang, Wei-Han Lee, Yueyao Chen, Chenyang Lyu, Chunming Wu, Raheem Beyah, Peng Cheng, Kangjie Lu, and Ting Wang. 2021. UNIFUZZ: A Holistic and Pragmatic Metrics-Driven Platform for Evaluating Fuzzers. In *USENIX Security*.

[26] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Vulnerability detection with fine-grained interpretations. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 292–303. https://doi.org/10.1145/3468264.3468597

[27] Zhen Li, Shouhuai Xu Deqing Zou and, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *NDSS*. https://doi.org/10.14722/ndss.2018.23158

[28] Ziyang Li, Saikat Dutta, and Mayur Naik. 2024. LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. arXiv:2405.17238 [cs.CR]

[29] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem. In *ICSE*. https://doi.org/10.1145/3510003.3510142

[30] Filipe Marques, Mafalda Ferreira, André Nascimento, Miguel E Coimbra, Nuno Santos, Limin Jia, and José Fragoso Santos. 2025. Automated Exploit Generation for Node.js Packages. In *PLDI*. https://doi.org/10.1145/3729304

[31] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chan, and Haipeng Cai. 2023. VulGen: Realistic Vulnerability Generation Via Pattern Mining and Deep Learning. In *ICSE*. https://doi.org/10.1109/ICSE48619.2023.00211

[32] Yu Nong, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai. 2022. Generating Realistic Vulnerabilities via Neural Code Editing: An Empirical Study. In *ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://doi.org/10.1145/3540250.3549128

[33] Wanzong Peng, Lin Ye, Xuetao Du, Hongli Zhang, Dongyang Zhan, Yunting Zhang, Yicheng Guo, and Chen Zhang. [n. d.]. PwnGPT: Automatic Exploit Generation Based on Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Vienna, Austria, 2025-07), Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 11481–11494. https://doi.org/10.18653/v1/2025.acl-long.562

[34] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (Jan. 2024), 85–105. https://doi.org/10.1109/TSE.2023.3334955

[35] Adriana Sejfia and Max Schaefer. 2022. Practical Automated Detection of Malicious npm Packages. In *ICSE*. https://doi.org/10.1145/3510003.3510104

[36] Gabriel Sherman and Stefan Nagy. 2025. No Harness, No Problem: Oracle-guided Harnessing for Auto-generating C API Fuzzing Harnesses. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 775–775. https://doi.org/10.1109/ICSE55347.2025.00239

[37] Cristian-Alexandru Staicu and Michael Pradel. 2018. Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers. In *USENIX Security Symposium*. 361–376.

[38] Cristian-Alexandru Staicu, Michael Pradel, and Ben Livshits. 2018. Understanding and Automatically Preventing Injection Attacks on Node.js. In *Network and Distributed System Security Symposium (NDSS)*.

[39] Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow Analysis-Inspired Deep Learning for Efficient Vulnerability Detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13. https://doi.org/10.1145/3597503.3623345

[40] Nikos Vasilakis, Cristian-Alexandru Staicu, Grigoris Ntousakis, Konstantinos Kallas, Ben Karel, André DeHon, and Michael Pradel. 2021. Preventing Dynamic Library Compromise on Node.js via RWX-Based Privilege Reduction. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 1821–1838. https://doi.org/10.1145/3460120.3484535

[41] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 819–831. https://doi.org/10.1145/3650212.3680323

[42] Jiacen Xu, Jack W. Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li. 2024. AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks. https://doi.org/10.48550/arXiv.2403.01038 arXiv:2403.01038 [cs]

[43] Soufian El Yadmani, Robin The, and Olga Gadyatskaya. 2023. Beyond the Surface: Investigating Malicious CVE Proof of Concept Exploits on GitHub. https://doi.org/10.48550/arXiv.2210.08374 arXiv:2210.08374 [cs]

[44] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). https://doi.org/10.52202/079017-1601

[45] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What Are Weak Links in the Npm Supply Chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340. https://doi.org/10.1145/3510457.3513044 arXiv:2112.10165 [cs]

[46] Michal Zalewski. 2013. American Fuzzy Lop (AFL). https://lcamtuf.coredump.cx/afl/. https://lcamtuf.coredump.cx/afl/

[47] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *ISSTA*. https://doi.org/10.1145/3650212.3680384

[48] Ying Zhang, Wenjia Song, Zhengjie Ji, Danfeng, Yao, and Na Meng. 2023. How Well Does LLM Generate Security Tests? https://doi.org/10.48550/arXiv.2310.00710 arXiv:2310.00710 [cs]

[49] Yuntong Zhang, Jiawei Wang, Dominic Berzin, Martin Mirchev, Dongge Liu, Abhishek Arya, Oliver Chang, and Abhik Roychoudhury. 2024. Fixing Security Vulnerabilities with AI in OSS-Fuzz. arXiv:2411.03346 [cs.CR] https://arxiv.org/abs/2411.03346

[50] Zenong Zhang, Zach Patterson, Michael Hicks, and Shiyi Wei. 2022. FixReverter: A Realistic Bug Injection Methodology for Benchmarking Fuzz Testing. In *Proceedings of the 31st USENIX Security Symposium*.

[51] Yunhui Zheng, Saurabh Pujar, Burn L. Lewis, Luca Buratti, Edward A. Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. 2021. D2A: A Dataset Built for AI-Based Vulnerability Detection Methods Using Differential Analysis. In *43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2021, Madrid, Spain, May 25-28, 2021*. IEEE, 111–120. https://doi.org/10.1109/ICSE-SEIP52600.2021.00020

[52] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the Npm Ecosystem. In *28th USENIX Security Symposium (USENIX Security 19)*. 995–1010.

Received 2025-09-12; accepted 2026-03-24